

HOMWORK-2

1. Define an integer array. Declare an integer pointer and initialize it to pointer to first element of array. Use Pointer arithmetic to transverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

```
#include <stdio.h>

int main() {

    int a[100], *ptr, i, n;

    printf("Enter n:");

    scanf("%d", &n);

    for(i=0; i<n; i++){

        printf("Value:");

        scanf("%d", &a[i]);

    }

    ptr=a;

    for(i=0; i<n; i++){

        printf("%d", *ptr);

        ptr++;

    }

    return 0;

}
```

OUTPUT:

```
Enter n:5
Value:1
Value:2
Value:3
Value:4
Value:5
12345

=== Code Execution Successful ===
```

2.Calculate the string length using pointers.

```
#include <stdio.h>
```

```
int main() {
    char a[100];
    char *ptr;
    printf("Enter the string:");
    scanf("%49[^\n]",a);
    ptr=a;
    while(*ptr!='\0'){
        ptr++;
    }
    printf("Length:%d",(int)(ptr-a));
    return 0;
}
```

OUTPUT:

```
Enter the string:VIVEKASHREE
Length:11

=== Code Execution Successful ===
```

3. Write a function that calculate both the sum and product of two numbers. Return these two results using pointers passed as arguments to the functions.

```
#include <stdio.h>

int calculate(int *a,int *b,int *sum,int *product){
    *sum=*a+ *b;
    *product=(*a)*( *b);
    return 0;
}

int main() {
    int a,b;
    int sum=0,product=0;
    printf("Enter a:");
    scanf("%d",&a);
    printf("Enter b:");
    scanf("%d",&b);
    calculate(&a,&b,&sum,&product);
    printf("Sum=%d\n",sum);
    printf("Product=%d",product);
    return 0;
}
```

OUTPUT:

```
Enter a:5
Enter b:2
Sum=7
Product=10

=== Code Execution Successful ===
```

4.Create an array of structures >Use dynamic memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

```
#include <stdio.h>

#include<stdlib.h>

struct student{

    int id;

    char name[20];

    float mark;

};

int main() {

    int n,i;

    printf("Enter n:");

    scanf("%d",&n);

    struct student *s;

    s=(struct student*)malloc(n*sizeof(struct student));

    for(i=0;i<n;i++){

        printf("Enter the ID:");

        scanf("%d",&(s+i)->id);

        printf("Enter the name:");

        scanf("%s",(s+i)->name);
```

```

    printf("Enter the marks:");
    scanf("%f",&(s+i)->mark);
}
for(i=0;i<n;i++){
    printf("%d\n %s\n %f\n",(s+i)->id,(s+i)->name,(s+i)->mark);
}
free(s);
return 0;
}

```

OUTPUT:

```

Enter n:2
Enter the ID:1
Enter the name:viveka
Enter the marks:98
Enter the ID:2
Enter the name:naresh
Enter the marks:100
1
viveka
98.000000
2
naresh
100.000000

=== Code Execution Successful ===

```

5. Define two functions with same signature. Declare a function pointer and use the pointer to call both function.

```

#include <stdio.h>

int sum(int a,int b){

```

```

    int sum;

    sum=a+b;

    return sum;
}

int difference(int a,int b){

    int difference;

    difference=a-b;

    return difference;
}

int main(){

    int a,b;

    printf("Enter a:");

    scanf("%d",&a);

    printf("Enter b:");

    scanf("%d",&b);

    int (*ptr)(int,int);

    ptr=sum;

    printf("Sum:%d\n",sum(a,b));

    ptr=sub;

    printf("Subtract:%d",difference(a,b));

    return 0;
}

```

OUTPUT:

```
Output
Enter a:5
Enter b:
2
Sum:7
Subtract:3

=== Code Execution Successful ===
```

6. Declare an integer pointer to it and a double pointer .Print the values and addresses at each level.

```
#include <stdio.h>
```

```
int main() {
```

```
int a,*p,**q;
```

```
printf("Enter the value: ");
```

```
scanf("%d",&a);
```

```
p=&a;
```

```
q=&p;
```

```
printf("%d\n",a);
```

```
printf("%p\n",*p);
```

```
printf("%p\n",**q);
```

```
printf("%d\n",&a);
```

```
printf("%p\n",&p);
```

```
printf("%p\n",&q);
```

```
return 0;
```

```
}
```

OUTPUT:

```
Enter the value: 1
1
0x1
0x1
-1107989236
0x7ffffbdf56d00
0x7ffffbdf56cf8

=== Code Execution Successful ===
```

7.Sort an array of floating point numbers using selection sort.

```
#include <stdio.h>
```

```
int main(){
```

```
    float a[100],temp;
```

```
    int i,n,j,min;
```

```
    printf("Enter n:");
```

```
    scanf("%d",&n);
```

```
    printf("Values:");
```

```
    for (i=0;i<n;i++){
```

```
        scanf("%f",&a[i]);
```

```
    }
```

```
    for(i=0;i<n-1;i++){
```

```
        min=i;
```

```
        for(j=i+1;j<n;j++){
```

```
            if(a[j]<a[min])
```

```
                min=j;
```



```

    }
    temp=a[i];
    a[i]=a[min];
    a[min]=temp;
}
for(i=0;i<n;i++){
    printf("%.2f\n",a[i]);
}
return 0;
}

```

OUTPUT:

```

Enter n:5
Values:9.8
8.7
7.6
5.4
4.3
4.30
5.40
7.60
8.70
9.80

=== Code Execution Successful ===

```

8. Define a structure ,student with id,name and grade.Create an array of students structures and

Sort them based on different features.

#include <stdio.h>

```
struct student{
    int id,marks;
    char name[50];
};
int main(){
    int n,a;
    struct student s[10],temp;
    int i,j;
    printf("N:");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Id:");
        scanf("%d",&s[i].id);
        printf("Name:");
        scanf("%s",s[i].name);
        printf("marks:");
        scanf("%d",&s[i].marks);
    }
    printf("Sort By:");
    printf("1.Id\n2.Marks\n3.Name");
    scanf("%d",&a);
    if(a==1){
        for(i=0;i<n;i++){
            for(j=i+1;j<n;j++){
                if(s[i].id>s[j].id){
                    temp=s[i];
```

```
        s[i]=s[j];
        s[j]=temp;
    }
}
}
}
if(a==2){
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(s[i].name>s[j].name){
                temp=s[i];
                s[i]=s[j];
                s[j]=temp;
            }
        }
    }
}
if(a==3){
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(s[i].marks>s[j].marks){
                temp=s[i];
                s[i]=s[j];
                s[j]=temp;
            }
        }
    }
}
```

```

    }
}
printf("\nSorted Students:\n");
for(i=0;i<n;i++){
    printf("%d %s %d\n",s[i].id,s[i].name,s[i].marks);
}
return 0;
}

```

OUTPUT:

The screenshot shows a dark-themed output window with the following text:

```

N:2
Id:1
Name:viveka
marks:98
Id:2
Name:naresh
marks:100
Sort By:1.Id
2.Marks
3.Name1

Sorted Students:
1 viveka 98
2 naresh 100

=== Code Execution Successful ===

```

9.Sort a set of negative number using radix sort.

```
#include <stdio.h>
```

```
void radixSort(int a[], int n)
{
    int output[100];
    int count[10];
    int max = a[0];
    int exp,i;

    for(i=1;i<n;i++)
    {
        if(a[i]>max)
            max=a[i];
    }

    for(exp=1;max/exp>0;exp*=10)
    {
        for(i=0;i<10;i++)
            count[i]=0;

        for(i=0;i<n;i++)
            count[(a[i]/exp)%10]++;

        for(i=1; i<10;i++)
            count[i]+=count[i-1];

        for(i=n-1;i>=0;i--)
        {
```

```

        output[count[(a[i]/exp)%10]-1]=a[i];
        count[(a[i]/exp)%10]--;
    }
    for(i=0;i<n;i++)
        a[i]=output[i];
    }
}

int main()
{
    int a[100],n,i;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter negative numbers:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
        a[i] =-a[i];
    radixSort(a,n);
    printf("Sorted negative numbers:\n");
    for(i=n-1;i>=0;i--)
        printf("%d ",-a[i]);
    return 0;
}

```

OUTPUT:

```
Enter number of elements: 5
Enter negative numbers:
-5
-7
-1
0
-6
Sorted negative numbers:
-7 -6 -5 -1 0

=== Code Execution Successful ===
```

10. Write a C program to copy one array to another using pointer.

```
#include <stdio.h>
```

```
int main(){
```

```
    int a[100], cpy[100], *ptr;
```

```
    int n, i;
```

```
    printf("N:");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++){
```

```
        scanf("%d", &a[i]);
```

```
    }
```

```
    ptr=a;
```

```
    for(i=0; i<n; i++){
```

```
        cpy[i]=(*ptr+i);
```

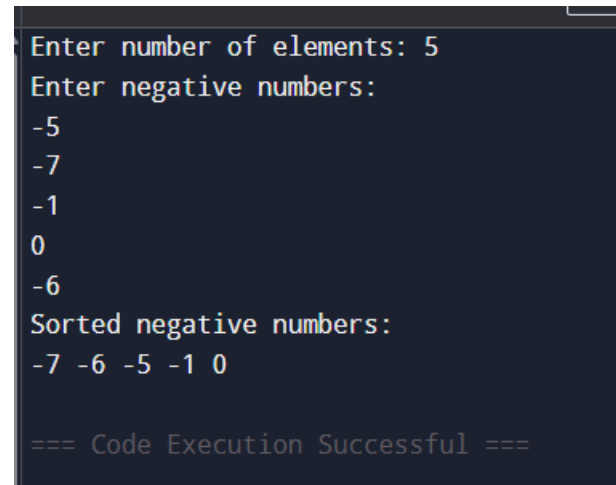
```
    }
```

```
    printf("Copied array:");
```

```
    for(i=0; i<n; i++){
```

```
        printf("%d ",cpy[i]);  
    }  
    return 0;  
}
```

OUTPUT:



```
Enter number of elements: 5  
Enter negative numbers:  
-5  
-7  
-1  
0  
-6  
Sorted negative numbers:  
-7 -6 -5 -1 0  
  
=== Code Execution Successful ===
```
